

Problem 1 — the market-listing trust layer

Approach note · Flent FDE take-home · deal FLT-FDE-2026-01

Every number here is printed by `trust-layer/trust_layer.py` running against the case packet; none were transcribed by hand. Assumptions are labelled as such.

1 · The problem, in my words

BOSS has to say whether a home is worth signing. One input is a market rent benchmark drawn from 86 scraped listings containing aliases, cross-posts, dead rows, bait, and — the one that matters — an inconsistent rent convention.

Who feels it. The acquisition lead owns the accept/negotiate/hold call and has to defend it; Supply carries it into a negotiation; the P&L carries it last. Ten extra vacant days push breakeven back a month, and here a ₹5,000/month swing in owner cost moves simple payback from 27.7 to 35.1 months.

The cost is not symmetric. Too low kills a good home — invisible, because nobody audits the deals you passed on. Too high signs a bad one — expensive, and blamed on demand rather than on the number. Both look identical on a dashboard, which is the brief's own sentence.

The framing I'd change. The brief describes this as a cleaning problem. I built the cleaner first, and measured it:

stage	rows	cut	median
raw pull	86		₹59,000
– broken / impossible	82	4	₹59,000
– cross-post duplicates	69	13	₹58,500
– stale	61	8	₹58,000
<i>above: not evidence · below: not comparable</i>			
– out of scope (BHK / society / furnishing)	20	41	₹59,500

86 rows to 20. The median moved ₹500. Cleaning did not fix the number, it collapsed the sample — and of the 66 rows removed only 25 were not evidence. The other 41 were good listings about a different kind of home. Merging those two categories would let me claim credit for removing far more junk than I did.

So cleaning is not the deliverable. Two things actually threaten this benchmark: **sample collapse** (the brief's own "30 comparables can suddenly have four") and **unit ambiguity** — comps not measuring the same quantity as the subject. Neither is a dirt problem, and neither is fixed by filtering harder.

Restated: the layer's job is not a cleaner rate. It is to decide *whether BOSS is allowed to speak at all*, and to make that decision inspectable.

2 · Approach, and what I chose not to solve

The pipeline never deletes. All 86 rows land in `out/ledger.csv` with a status, a rule id, and a sentence a non-engineer can argue with. An exclusion is a claim to be defended, not a row that vanishes.

Confidence is not a function of n . The one structural thing I'd insist on. v1 graded confidence on sample size — and sample size is what *every* defect inflates. Duplicates raise n ; bait tightens the cluster; wrongly merged societies enlarge the comp set. Surviving junk was buying confidence, the exact inversion of what a trust layer is for.

v2 asks not "*how much evidence is there?*" but "**how hard is this call to overturn?**" The engine runs ~20 attacks (dedup failing entirely, the k cheapest rows being bait, the dominant portal removed, worst-case subset deletion, ask-inflation swept 2–10%) and reports how many the call survives. A verdict that holds when you assume every defence failed deserves trust no sample size can justify on its own.

It refuses on the action, not the label. AT MARKET and BELOW MARKET are different words for the same decision; only a *negotiate* versus *no objection* split costs anyone anything, so only that triggers a refusal. A system that refuses whenever anything wobbles is hedging.

What I deliberately did not build:

- **The rest of the deal.** Capex (₹1.6L placeholder, no BOQ), tenant revenue (₹72k hypothesis, no signed tenants) and fill time are larger uncertainties than the benchmark. I did not model any of them. The site *displays* them, recomputed from the assessment's own inputs and tagged with how solid each is (§6b), because a market rate means nothing next to a cost you cannot see. Judging them is a different problem. §5 shows one of them is in fact the bigger error.

- **A metrics dashboard.** The output here is a decision and one question, not a screen of numbers. The surface I did build has no aggregate market views: anything inviting a rate to be read off a screen without its ledger reintroduces the failure this project exists to fix.
- **Any model.** Nothing here needs one, and a model would have made the exclusions unarguable.
- **Photo/text dedup and calibration.** No comp images and no signed rents in the packet. Both are named as dependencies rather than faked.

3 · The shape of the system

The dividing line I'd hold: **AI proposes structure, deterministic code decides, humans own the boundaries.**

Deterministic — everything that removes or weighs evidence. Filters, thresholds, the benchmark, the interval, the verdict, the robustness battery. Seeded, so it answers the same way twice. Every exclusion must reduce to one sentence with a rule id that a Supply reviewer can disagree with; anything that can't doesn't belong here.

AI — only where input is unstructured and output is a *proposal*, never a decision. Three places, all upstream of the arithmetic: society name clustering (16 spellings → 5 societies, no hand-typed alias table); description and photo similarity for content-based dedup, the real fix for cross-posts, which currently rest on a date tolerance; and extracting basis declarations from free text ("rent inclusive of maintenance", "carpet area") — the fork in §4 is a *reading* problem, which is what an LLM is for.

None touch the median. Each writes to a review file (`out/alias_registry.csv`) a human confirms, because the failure mode is silent: merging two genuinely different societies produces a *larger* comp set, which reads as better evidence. The shipped clustering is plain string similarity, not AI — the cheapest thing that worked, with the harder version left as a named upgrade.

Human — the boundaries, and the veto. Which societies and furnishing tiers count as comparable is a judgment about the market, not a computation, and so are the two thresholds deciding when BOSS may speak. Any row can be forced in or out via `overrides.csv` by a named person who outranks every rule in both directions, with the overridden rule left visible so the disagreement is recorded rather than erased.

The thresholds audit themselves. The engine refuses any listing arriving without a source and a freshness; holding its own numbers to a lower standard would be incoherent, and moving a number into a config block relocates it rather than justifying it. Each of the 10 real thresholds declares what set it and what breaks if it is wrong: **4 EVIDENCED, 6 ASSUMED**, printed on every run.

4 · Missing, stale, conflicting, unreliable

These are four different failures and they need four different instruments. Treating them alike is how a biased number gets a tight confidence interval.

	nature	instrument
Noisy	statistical	median + MAD (not mean/SD — outliers poison the statistic used to detect them); seeded bootstrap interval
Stale	decays	age filter — and shown here to be <i>irrelevant</i> : swept 14/21/30/45 days and no filter at all, median ₹59,500 in every case
Missing	not a measurement problem	refuse , and name the cheapest resolving question
Biased	invisible to any filter	bound it — sweep the defect and report where the call flips

The worked example — and the thing that decides this deal. Market ops flagged it on 18 Aug: "*we do not capture maintenance reliably, so the listing rents are not always all-in.*" The 20 surviving comps therefore mix two conventions, medianed as if they were one measurement. Against the same ₹59,500 benchmark:

	compare against	deviation	reading
if listings quote base rent	₹56,000	−₹3,500 (−5.9%)	below market — the ask is defensible
if listings quote all-in rent	₹61,000	+₹1,500 (+2.5%)	above market — we are overpaying

Same clean data, opposite verdicts, ₹5,000/month apart. **No further cleaning fixes this: the information was never scraped.** It is *missing*, not noisy, so a confidence interval is the wrong instrument — the uncertainty is not statistical. The engine therefore publishes the benchmark at MEDIUM and returns `INSUFFICIENT_EVIDENCE` on the deviation: two questions, answered separately. It refuses only because the two readings *straddle* the benchmark; where both land the

same side it answers.

The deliverable is not the refusal — it's the resolving action attached to it:

Supply, one question to the landlord, answerable in a day: *is the ₹5,000 society maintenance included in the quoted rent, or charged on top?*

Bias, where refusal isn't available. A landlord asking ₹70,000 and settling at ₹62,000 leaves only ₹70,000 in the data. The obvious signal — an aspirational ask should sit unsold longer — gives `correlation(days on market, rent) = -0.16`: wrong sign, negligible. **Nothing in a listings-only dataset separates an aspirational ask from a real one.** It can't be filtered, so it is bounded: the call is stable to 6% systematic ask-inflation and flips at 8%. An unmeasurable risk becomes a stated threshold.

The subject side is an assumption too. Every filter above tests the *comps*, but which 20 rows are comparable at all depends on the subject's own furnishing tier — recorded by Supply, not measured, and furnishing is the largest scope filter here (within Lakeview: ₹54,500 unfurnished → ₹59,500 semi → ₹68,500 fully, ~24%). Were the subject fully furnished, this deal would be the $n=4$ refusal case. The packet's four site-visit stills are the only independent check and they hold: a built-in wardrobe, no bed or soft furnishing, matching the recorded tier. Corroboration, not measurement — but I checked the assumption the whole comp set hangs on rather than inheriting it from a form field.

Never invent. Blank fields stay blank, an unrecognised society name is kept verbatim rather than guessed into a cluster, and a deal missing a required term is rejected rather than defaulted. The one synthetic thing in the build, two fabricated rows demonstrating the `min_n` cliff, is labelled as fabricated.

And the failure case is real, not staged. Same pipeline, same thresholds, no special-casing — the same deal record with furnishing changed to fully furnished. Not a second property; the packet contains one deal and this is it, one configuration to the left: $n=4$, `INSUFFICIENT EVIDENCE`, nothing published. The brief's own "30 comparables can suddenly have four", reproduced. The run then names the two ways a human could resolve it, and that both are human choices.

5 · The first thing I'd validate

First I checked whether the packet could test the premise. It can. `outcomes.csv` is the only file here pairing a forecast with what actually happened, and I had not looked at it when I picked this problem. `trust-layer/outcomes_check.py` does not confirm the thesis:

forecast	median actual ÷ plan	worse than plan	sign test
Tenant revenue	0.98	6 of 10	$p = 0.75$
Days to fill	1.23	8 of 10	$p = 0.11$
Payback months	1.15	8 of 10	$p = 0.11$

The pricing is close to right. The timing is not. The overrun is not a flat tax: median 6 days late, mean 12, because two of ten ran 37 and 39 days over — ~3.7–3.9 months of payback each at the packet's own ten-days-per-month rate.

Flent's notes name both. The worst, at 2.16x, is a **clock-start error**: *"repair work delayed launch by 12 days; fill clock starts at handover in the forecast."* The forecast counts from handover; the home cannot let until the work is finished. That is not a modelling failure, it is counting from the wrong event. The second, at 1.89x, is a **definition error**: *"third room had no attached bathroom; forecast used the three-room average."* The packet's own vocabulary already separates a lettable room from an advertised bedroom, and the forecast ignored the distinction.

Neither is a pricing error, and neither needs a model. They are the same failure this layer addresses — a number that is wrong because nobody agreed what was being measured — pointed at a different input. The tail shape decides the rest: a systematic bias takes a multiplier, but a two-in-ten blowout does not. Adding 23% to every forecast would be wrong on the eight deals that landed near plan in order to be right on the two that did not.

Ten cases is not proof: $p = 0.11$ misses 5% significance, and excluding the still-filling case biases it optimistic, so the true miss is worse than 23%. Enough to reorder the work, not enough to act blind.

So: fill-time calibration first, ask-versus-signed second. The second is still the assumption *this layer* rests on — that a **benchmark built from asks stands in for achieved rent** — and it is load-bearing twice: the only unbounded defect in the engine (§4), and the two thresholds deciding when BOSS may speak are claims about *future accuracy*, which listings can never check however hard I clean. First inside Problem 1; not first at Flent.

The measurement. Join 50–100 of Flent's signed rents to the benchmark live at signing and plot `signed ÷ benchmark`. It **works** if the median ratio sits within $\pm 6\%$ and MEDIUM benchmarks are tighter than LOW; it **fails** at ~8% or worse, or if the tiers don't separate — in which case every benchmark in BOSS is biased high, the deviation

logic needs a correction factor rather than more filtering, and `min_n` becomes a graded floor from data instead of the least-defended number in the engine. The 6% is the flip point the battery already measured, so it is pre-registered rather than chosen after seeing the answer. Flent owns the data: a join, not a build.

Third, and cheapest: do refusals get resolved, or routed around? **Share of refusals closed by evidence within 3 days versus closed by override**, watched from day one. A trust layer whose refusals are overridden has added a step, not reduced risk.

6 · Assumptions, scope cuts, trade-offs

- **Near-symmetric noise is an assumption about this pull**, not a property of the market — it is why cleaning barely moves the centre, and real listing noise is one-directional by construction (§4). Labelled in the code.
- **Listings are asks, not transactions**. The benchmark is labelled a ceiling-biased estimate of asks, not of achieved rent.
- **Dedup is deliberately strict** — society, area, BHK, posted dates within 2 days, rent within 2%. The looser rule collapses 19 rows here, but 7 are distinct homes that merely share a floor area. Under-merging shrinks a comp set visibly; over-merging corrupts it invisibly, so I chose the recoverable error. *It cost me a better-looking result*: the loose rule gives $n=17$ and ₹58,500, exactly midway between the two maintenance readings. That symmetry was an artifact and is not in this submission.
- **Two findings were withdrawn after being wrong** (the equidistance above, and an overstated "removing any listing changes nothing"), documented in `FAILURES.md` rather than quietly deleted.
- **Nine defects found by attacking my own build: 3 fixed, 2 bounded, 4 open**. `FAILURES.md` is the honest version of this note, and the backlog the execution plan is drawn from.

The conclusion I didn't expect. Of the three fixes that would most improve this layer — a basis field on the scrape (rent all-in? area carpet or super built-up?), content-based dedup, and calibration against signed rents — **two are scraping changes, not modelling changes**. The best next move is capturing *more about each listing*, not filtering existing fields harder. The third is a hard dependency on Problem 2: this layer cannot be finished alone.

6b · What I actually shipped

The proof of work is a script and it is the thing to read. But a refusal is only useful if whoever must act on it can see it, so the layer is also deployed:

- **flent-trust-layer.vercel.app** — the deal, the rate, the refusal, and all 86 listings with the rule and sentence that excluded each. Any row can be opened and disagreed with, and recording the landlord's answer re-scores the deal, so the refusal has somewhere to go.
- **A Python API** importing `trust_layer.py` rather than reimplementing it, so the site cannot drift from the proof.
- **Upload your own pull** — a listings CSV and a deal record through the identical pipeline. A deal missing a required term is refused rather than defaulted.
- **Nine labelled synthetic scenarios**, two built to defeat the engine and saying so.
- **The money on the deal**, recomputed from the packet's inputs and asserted against the figures the assessment publishes, so a mistyped input surfaces as a mismatch rather than a plausible answer. **None of this is the trust layer's work** and the tab says so: it is BOSS's own arithmetic, shown beside the rate it is judging, with all four soft inputs labelled `hypothesis`, `placeholder` or `assumption`.
- `outcomes_check.py` — §5's run, which tests this submission's premise against Flent's own history and does not fully support it.

This is FLT-9 brought forward, scoped as a ledger and refusal surface rather than a dashboard. Built *first* it would have undercut the thesis, because the temptation would have been to make the number look good rather than the evidence arguable.

7 · How I used AI

Claude interrogated the framing before any code was written, scanned the 86 rows for alias and duplicate patterns, drafted the engine, and attacked it — the stress battery came from asking it how to make the layer confidently wrong, which is how the "confidence must not be a function of `n`" inversion surfaced. The judgment calls are mine: dedup strictness (which overturned an earlier, better-looking result), the MAD threshold at 3.5, holding refusal to *action* rather than label disagreements, and making a refusal and a null result the headline outputs rather than caveats.

Disagreements with the model's first answer are recorded in `FAILURES.md`.